ArquiKota — Documentação do projeto

Trabalho n.º 16 · ArquiKota · monitor de manutenção contínua Monitor de manutenção contínua para projetos Next.js/Vercel.

Data de referência: outubro de 2026. Aplicação: Next.js 16, React 19 e TypeScript, com Supabase, EmailJS, GitHub Actions e Vercel.

---

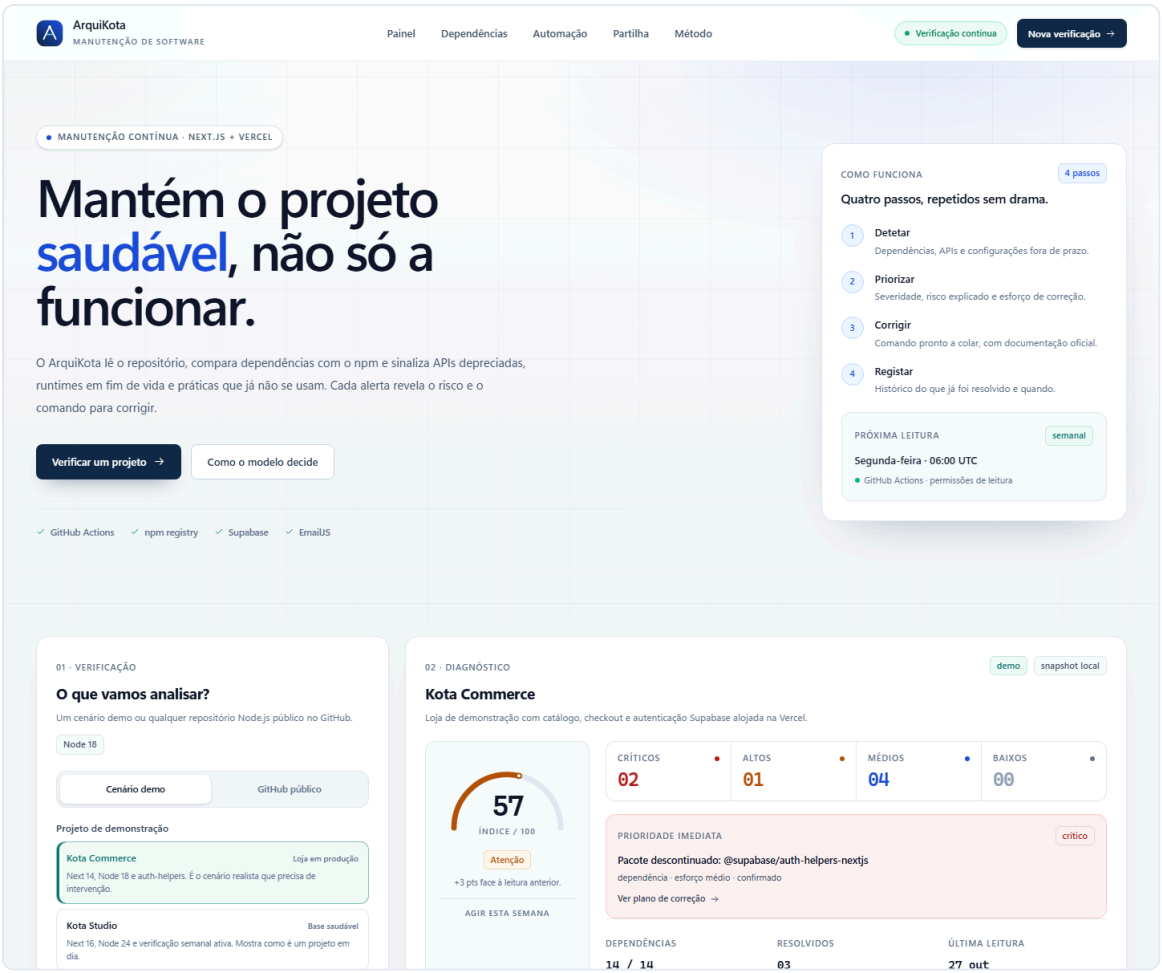
1. Tema e objetivo

O ArquiKota responde a uma pergunta prática: **o que está a envelhecer neste projeto e o que deve ser corrigido primeiro?**

A aplicação importa um repositório Node.js público do GitHub ou usa um cenário de demonstração, lê dependências e configuração, compara versões com o npm e aplica um catálogo de regras de manutenção. O resultado é um relatório com:

- alertas de dependências desatualizadas, runtimes em fim de vida, APIs depreciadas e práticas obsoletas;
- severidade, urgência, confiança e explicação do risco de ignorar cada alerta;
- comando ou snippet de correção pronto a executar e ligação à documentação oficial;
- índice de saúde de 0 a 100 e histórico de alertas resolvidos;
- possibilidade de enviar o resumo por email;
- automação agendada por GitHub Actions.

O objetivo pedagógico é mostrar que manutenção não é uma verificação única: é um ciclo de detetar, triar, corrigir e confirmar.



Painel do ArquiKota com diagnóstico, alertas e índice de saúde

2. Requisitos do trabalho e resposta do projeto

Requisito	Como foi cumprido
Detetar dependências desatualizadas	Comparação da versão instalada com a versão `latest` do npm; fallback para catálogo local quando o registry falha.
Detetar APIs depreciadas da Vercel/Supabase	Regras para `images.domains`, `@supabase/auth-helpers-nextjs` e migração para `@supabase/ssr`.
Detetar práticas obsoletas	Regras para Node em fim de vida, runtime desatualizado no `vercel.json`, ausência de lockfile, de testes e de verificação agendada.
Mostrar urgência	Severidade crítica/alta/média/baixa, esforço, confiança e índice de saúde.
Explicar o risco de ignorar	Campo “Se ignorares” em cada alerta, com consequência concreta.
Fornecer correções prontas	Comandos e snippets copiáveis, com texto de apoio e fonte oficial.
Histórico de alertas resolvidos	Persistência no Supabase, com cache local como reserva; cada entrada guarda título, categoria, data e nota.
Verificação periódica	Workflow `.github/workflows/arquikota-scan.yml`, semanal e manual, com relatório como artefacto.
Ensinar manutenção contínua	Secção de metodologia e ciclo de manutenção na página.
Stack obrigatória: Supabase	Histórico de resoluções persistido no Postgres do Supabase com RLS.
Stack obrigatória: EmailJS	Envio do resumo do diagnóstico por email a partir do servidor.
Documentação	Este documento, em Markdown, Word e PDF.

3. Stack tecnológica

Camada	Tecnologia	Uso no projeto
Framework	Next.js 16.4 (App Router)	Página, API Routes e metadados.
UI	React 19.3, Tailwind CSS 4.3	Painel responsivo com identidade corporativa clara: azul-marinho, superfícies brancas e hierarquia tipográfica sóbria.
Linguagem	TypeScript 5.9	Contratos de dados, motor de regras e validação.
Runtime	Node.js 24	Servidor Next e script de CI.
Base de dados	Supabase (Postgres)	Histórico de resoluções por projeto.
Email	EmailJS REST API	Envio do relatório por email.
Fonte de repositórios	GitHub REST API	Importação de `package.json`, configuração e workflows.
Versões	npm registry	Versão `latest` e estado de depreciação dos pacotes.
Automação	GitHub Actions	Verificação agendada e manual.
Alojamento	Vercel	Publicação da aplicação Next.js.
Testes	Vitest 5	36 testes unitários e de contrato.

4. Funcionalidades

4.1 Verificação de um projeto

O utilizador escolhe uma de duas origens:

- **Cenário demo**: Kota Commerce, Kota Studio ou Kota Legado, com dados realistas e determinísticos.
- **GitHub público**: qualquer repositório Node.js, indicado como `owner/repo` ou URL do GitHub.

Também escolhe a cadência desejada (semanal ou diária), que influencia o alerta de automação e o YAML sugerido.

4.2 Diagnóstico

Cada verificação produz:

- **Índice de 0 a 100**, calculado por `100 – soma dos pesos` dos alertas ativos.
- **Veredicto**: Saudável (75+), Atenção (55–74) ou Crítico (abaixo de 55).
- **Contagens por severidade** e alerta de prioridade imediata.

- **Dependências**: instalada, latest, diferença e severidade por número de versões major.
- **Histórico de score** por leitura, para mostrar tendência.

4.3 Alertas

Cada alerta inclui título, sujeito, versão atual e recomendada, resumo, risco, esforço, confiança, comando/snippet e ligação oficial. As regras atuais cobrem:

- Node.js em fim de vida (`engines` , `.nvmrc` e runtime da Vercel);
- Next.js desatualizado e migrações major relevantes;
- pacotes descontinuados, como `@supabase/auth-helpers-nextjs` ;
- APIs de configuração removidas, como `images.domains` ;
- dependências desatualizadas por diferença de versões;
- ausência de `package-lock.json` ;
- ausência de scripts de teste;
- ausência de verificação agendada.

As heurísticas são identificadas como tal; não são apresentadas como certezas.

4.4 Resolução e histórico

Ao marcar um alerta como resolvido, o score é recalculado imediatamente e a entrada aparece no histórico. O registo é enviado para `POST /api/resolutions` e guardado no Supabase. Se o Supabase estiver indisponível, a aplicação mantém o registo no `localStorage` e mostra o estado "modo local" em vez de esconder a falha.

O botão **Limpar histórico** apaga os registos do projeto, mediante confirmação.

4.5 Partilha por email

O painel "Partilha" envia um resumo com projeto, score, veredicto, número de alertas críticos/altos, alerta prioritário, data e link do painel. O envio é feito no servidor: a chave privada do EmailJS nunca chega ao browser.

4.6 Automação

O painel "Automação" mostra a cadência, a próxima execução, o caminho do workflow e o YAML copiável. O workflow incluído corre às segundas-feiras às 06:00 UTC e também pode ser executado manualmente. O script `npm run scan:ci` gera `maintenance-report.md` , publicado como artefacto durante 30 dias.

5. Arquitetura

```
Browser
  └─ Painel React (app/page.tsx, components/dashboard.tsx)
       ├─ POST /api/scan ──────────► Motor determinístico (lib/engine.ts)
       │                              ├─ Presets demo (lib/presets.ts)
       │                              ├─ GitHub REST (lib/github.ts)
       │                              └─ npm registry (lib/registry.ts)
       ├─ GET/POST/DELETE /api/resolutions
       │                              └─ Supabase (lib/supabase.ts, lib/resolutions.ts)
       └─ POST /api/notify ──────► EmailJS REST (lib/email.ts)
```

Princípios:

- **Servidor apenas para segredos**: `SUPABASE_SERVICE_ROLE_KEY` e `EMAILJS_PRIVATE_KEY` não têm prefixo `NEXT_PUBLIC_`.
- **Determinismo**: as mesmas entradas produzem o mesmo relatório; não há chamadas a modelos de IA.
- **Degradação explícita**: falhas do npm, GitHub, Supabase ou EmailJS são comunicadas, não silenciadas.
- **Validação na fronteira**: todos os pedidos são validados em contratos próprios (`lib/*-contract.ts`).

6. API

Método	Rota	Função	Proteções
POST	`/api/scan`	Verifica demo ou repositório GitHub.	Corpo máximo 8 KB, validação de origem/repo, sem escrita no GitHub.
GET	`/api/resolutions?projectId=...`	Lista o histórico do projeto.	ID normalizado; resposta `no-store`.
POST	`/api/resolutions`	Cria ou atualiza uma resolução.	Corpo 4 KB, validação estrita, 30 escritas/10 min/IP.
DELETE	`/api/resolutions?projectId=...`	Limpa o histórico do projeto.	Corpo 4 KB, 30 escritas/10 min/IP.
POST	`/api/notify`	Envia o resumo por email.	Corpo 4 KB, validação, 5 envios/10 min/IP.

Todas as respostas de erro usam JSON com uma mensagem em português e não expõem detalhes internos.

7. Supabase

7.1 Tabela

A migração `supabase/migrations/20261025000000_arquikota_resolutions.sql` cria a tabela `public.arquikota_resolutions`:

- chave primária composta por `project_id` e `alert_id`;
- categoria limitada a runtime, dependency, platform, practice ou automation;
- nota, data de resolução e data de criação;
- índice por projeto e data;
- **Row Level Security ativada sem políticas**, pelo que só a service role do servidor consegue ler ou escrever.

7.2 Como aplicar

1. Abrir o projeto no Supabase.
2. Ir a **SQL Editor**.
3. Colar o conteúdo do ficheiro de migração e executar.
4. Confirmar em **Table Editor** que existe `arquikota_resolutions`.
5. Recarregar o painel: o selo do histórico deve passar de "LOCAL" para "SUPABASE".

Enquanto a tabela não existir, a API responde `503` com uma mensagem a indicar a migração, e o painel continua a funcionar em modo local.

7.3 Variáveis

```
SUPABASE_URL=https://SEU-PROJETO.supabase.co
SUPABASE_SERVICE_ROLE_KEY=...
```

A URL pode incluir `/rest/v1/`; o servidor normaliza para a raiz. A chave service role é secreta e só deve existir no servidor e nas variáveis do alojamento.

8. EmailJS

8.1 Variáveis

```
EMAILJS_SERVICE_ID=...
EMAILJS_TEMPLATE_ID=...
EMAILJS_PUBLIC_KEY=...
EMAILJS_PRIVATE_KEY=...
```

8.2 Template

Antes de testar, confirmar em **Account** → **Security** que o acesso à API para aplicações não-browser está ativo e que a opção de chave privada está ligada. Sem esta autorização, o servidor recebe uma recusa

mesmo com todas as chaves corretas.

No template, o campo **To Email** tem de ser exatamente `{{to_email}}`. Se estiver vazio ou usar outro nome, como `{{email}}`, o EmailJS responde com erro de destinatário.

Assunto sugerido:

```
[ArquiKota] {{project_name}} · {{score}}/100 · {{status}}
```

Conteúdo sugerido:

```
<h2>Relatório ArquiKota</h2>
<p><strong>Projeto:</strong> {{project_name}}</p>
<p><strong>Índice:</strong> {{score}}/100 · {{status}}</p>
<p><strong>Alertas:</strong> {{critical_count}} críticos, {{high_count}} altos ·
        {{total_alerts}} no total</p>
<p><strong>Prioridade:</strong> {{top_alert}}</p>
<p><strong>Leitura:</strong> {{scanned_at}}</p>
<p><a href="{{panel_url}}">Abrir painel</a></p>
```

O template deve incluir as variáveis:

Variável	Conteúdo
`to_email`	Destinatário indicado no painel.
`project_name`	Nome do projeto analisado.
`score`	Índice de 0 a 100.
`status`	Saudável, Atenção ou Crítico.
`critical_count`	Número de alertas críticos.
`high_count`	Número de alertas altos.
`total_alerts`	Total de alertas ativos.
`top_alert`	Título do alerta prioritário (ou estado limpo).
`scanned_at`	Data da leitura.
`panel_url`	Link para o painel.

Para validar o template sem passar pela aplicação, usar o separador **Test it** do EmailJS com estes valores de exemplo:

Campo	Valor
`to_email`	O teu próprio email.
`project_name`	Kota Commerce
`score`	57
`status`	Atenção
`critical_count`	2
`high_count`	1
`total_alerts`	7
`top_alert`	Pacote descontinuado: @supabase/auth-helpers-nextjs
`scanned_at`	26/10/2026 11:30 UTC
`panel_url`	http://localhost:3000/#diagnostico

Configurar o campo **To Email** do template como `{{to_email}}`. O envio usa a API REST oficial com chave privada e um timeout de 12 segundos. Se o EmailJS recusar, a aplicação mostra o código HTTP devolvido e a indicação do que confirmar; o motivo detalhado fica no log do servidor.

8.3 Limites

Cada IP pode pedir 5 envios em 10 minutos. O servidor valida o email, o score, o veredicto e a coerência das contagens antes de contactar o EmailJS.

9. Instalação local

Pré-requisitos: Node.js 22 ou superior e npm.

```
npm install
cp .env.example .env.local   # preencher os valores reais
npm run dev
```

Abrir `http://localhost:3000`.

Comandos úteis:

```
npm test          # 36 testes
npm run typecheck # TypeScript
npm run build     # build de produção
npm start         # servir o build
npm run scan:ci   # gerar maintenance-report.md
```

O ficheiro `.env*` está ignorado pelo Git; `.env.example` não contém segredos.

10. Deploy na Vercel

1. Publicar o projeto num repositório GitHub (o ficheiro `.gitignore` já exclui `.env.local`).
2. Importar o repositório na Vercel.
3. Adicionar as variáveis de ambiente de `.env.example` em **Settings → Environment Variables**, incluindo `SUPABASE_SERVICE_ROLE_KEY` e `EMAILJS_PRIVATE_KEY`.
4. Definir `NEXT_PUBLIC_SITE_URL` com o domínio final (opcional; sem ela é usado o origin do pedido).
5. Fazer deploy. O build usado é `npm run build`.
6. Aplicar a migração SQL no Supabase, se ainda não foi feita.
7. Testar no domínio público: verificação demo, importação GitHub, resolução, recarregamento e envio de email para um endereço próprio.

Segredos não devem ser colocados em repositório, documentação ou capturas de ecrã.

11. Segurança e privacidade

- Importação apenas de repositórios públicos e **apenas leitura**; o ArquiKota não escreve no GitHub.
- Validação de referências GitHub para impedir URLs de outros domínios (proteção contra SSRF).
- Limite de tamanho do corpo em todas as rotas.
- Rate limiting em memória para resoluções e envios de email.
- Validação estrita de email, categorias, IDs, títulos e notas.
- Supabase com RLS ativa e sem políticas; acesso exclusivo do servidor.
- Chaves privadas apenas em variáveis de ambiente do servidor.
- Nenhum conteúdo do repositório é enviado para serviços de IA.
- Dependências de produção sem vulnerabilidades conhecidas no momento da verificação (`npm audit`).

Limitações: o rate limit em memória é adequado a uma instância; em produção multi-instância deve ser substituído por um store partilhado. O `GITHUB_TOKEN` é opcional e aumenta o limite da API pública.

12. Cenários de demonstração

Cenário	Perfil	Score	Leitura
Kota Commerce	Next 14, Node 18, auth-helpers	57 · Atenção	Caso realista com dívida acumulada e alertas críticos.
Kota Studio	Next 16, Node 24, verificação ativa	100 · Saudável	Projeto em dia.
Kota Legado	Next 12, Node 16, configuração antiga	46 · Crítico	Pior caso, com várias ruturas prováveis.

Os cenários usam um snapshot de versões datado de outubro de 2026. Repositórios reais consultam o npm em tempo real; se o registry falhar, o resultado indica “snapshot local” e lista o aviso.

13. Testes e verificação

Verificações efetuadas:

- **36 testes Vitest** a passar: motor de regras, validação de pedidos, contratos de resolução, contratos de email e presets.
- **TypeScript** sem erros (`tsc --noEmit`).
- **Build de produção** Next.js concluído com as rotas `/` , `/api/scan` , `/api/resolutions` e `/api/notify` .
- **UI desktop e mobile**: 390 px sem scroll horizontal, painéis de histórico e partilha visíveis.
- **API demo**: Kota Commerce devolve 57, com alertas e plano de correção.
- **Importação GitHub**: fluxo testado com repositório público.
- **Fluxo de resolução**: score sobe após resolução e regressa ao limpar.
- **Fallback local**: com Supabase indisponível, o histórico mantém-se e o erro é visível.
- **Automação**: YAML copiável e `scan:ci` gera o relatório.

O envio real de email deve ser testado pelo utilizador com um endereço próprio, para não enviar mensagens a terceiros durante o desenvolvimento.

14. Limites do modelo

O ArquiKota não substitui `npm audit` , testes de integração ou revisão humana. Não procura vulnerabilidades no código da aplicação; observa dependências, configuração e práticas declaradas no repositório. Versões e cronogramas mudam, pelo que cada correção deve ser confirmada na fonte oficial.

15. Deliverables

- [x] Aplicação funcional local.
- [x] Documentação em Markdown, Word e PDF.
- [x] Migração Supabase.
- [x] Workflow GitHub Actions.
- [ ] Repositório GitHub público.
- [ ] URL pública na Vercel.
- [ ] Migração SQL aplicada no Supabase e teste de email com endereço próprio.

16. Estrutura do projeto

```
 app/
   api/scan/route.ts
   api/resolutions/route.ts
   api/notify/route.ts
   page.tsx
components/        painel, alertas, histórico, partilha, metodologia
lib/               motor, presets, GitHub, npm, Supabase, EmailJS, contratos
scripts/scan-ci.ts relatório de CI
supabase/migrations/
.github/workflows/arquikota-scan.yml
tests/             36 testes
```